Chapitre 6 : Graphiques Avancés avec D3.js

Par Robert DIASSÉ

Objectifs :

1. Créer des graphiques plus complexes (cartes, diagrammes de Sankey, graphiques à bulles).
2. Comprendre les techniques avancées de mise en page.
3. Résoudre des problèmes typiques rencontrés dans les visualisations avancées avec D3.js.

1. Graphiques à Nuages de Points (Scatter Plots) Avancés avec Interactions

Les graphiques à nuages de points sont utilisés pour afficher des relations entre deux ou plusieurs variables. Les versions avancées peuvent inclure des interactions comme le survol des points ou l'intégration avec des graphiques d'autres types, comme les graphiques linéaires.

Propriétés des Graphiques à Nuages de Points :

- `cx`, `cy` : Les coordonnées du centre des points.
- `r` : La taille du point (souvent utilisée pour représenter une autre dimension de la donnée).
- `color` : La couleur du point, souvent liée à une dimension catégorique.

Exemple de Graphique à Nuage de Points Avancé avec Interactions :

```
<svg width="600" height="400"></svg>
<script>
  const data = [
    { x: 30, y: 30, size: 10, color: 'red' },
    { x: 70, y: 80, size: 20, color: 'blue' },
    { x: 110, y: 50, size: 15, color: 'green' },
    { x: 150, y: 90, size: 25, color: 'yellow' },
  ];

  const svg = d3.select("svg");

  const xScale = d3.scaleLinear().domain([0, 200]).range([0, 600]);
  const yScale = d3.scaleLinear().domain([0, 200]).range([400, 0]);
```

```
  svg.selectAll("circle")
    .data(data)
    .enter()
    .append("circle")
    .attr("cx", d => xScale(d.x))
    .attr("cy", d => yScale(d.y))
    .attr("r", d => d.size)
    .attr("fill", d => d.color)
    .on("mouseover", function(event, d) {
      d3.select(this).attr("fill", "orange");
      // Afficher une info-bulle avec les détails
      svg.append("text")
        .attr("x", xScale(d.x) + 10)
        .attr("y", yScale(d.y) - 10)
        .text(`X: ${d.x}, Y: ${d.y}`);
    })
    .on("mouseout", function() {
      d3.select(this).attr("fill", d => d.color);
      svg.selectAll("text").remove();
    });
</script>
```

---

2. Diagrammes de Boîte (Boxplots)

Les diagrammes de boîte sont utilisés pour résumer la distribution des données à travers leurs quartiles, et identifier les valeurs aberrantes.

Propriétés des Diagrammes de Boîte :

- `d3.boxPlot()` : Définit les données pour les diagrammes de boîte.
- `min`, `max`, `Q1`, `Q3`, `median` : Représentent respectivement les valeurs minimales, maximales, les premier et troisième quartiles et la médiane.

Exemple de Diagramme de Boîte :

```
<svg width="600" height="400"></svg>
<script>
  const data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

  const svg = d3.select("svg");
  const boxWidth = 50;

  const boxPlot = d3.boxPlot()
    .domain([0, d3.max(data)])
    .width(boxWidth);

  svg.append("g")
    .datum(data)
```

```
    .call(boxPlot);
</script>
```

---

3. Graphiques de Séries Temporelles Avancées (Time Series Graphs)

Les graphiques de séries temporelles sont utilisés pour afficher des données chronologiques. Dans une version avancée, on peut inclure des annotations, des intervalles de confiance, ou encore des visualisations multiples avec des axes secondaires.

Propriétés des Séries Temporelles Avancées :

- `d3.line()` : Trace les lignes représentant l'évolution dans le temps.
- `d3.scaleTime()` : Crée des échelles adaptées aux données temporelles.

Exemple de Graphique de Séries Temporelles :

```
<svg width="800" height="400"></svg>
<script>
  const data = [
    { date: new Date(2021, 0, 1), value: 10 },
    { date: new Date(2021, 1, 1), value: 20 },
    { date: new Date(2021, 2, 1), value: 15 },
    { date: new Date(2021, 3, 1), value: 25 },
  ];

  const svg = d3.select("svg");

  const xScale = d3.scaleTime().domain([new Date(2021, 0, 1), new Date(2021, 3,
1)]).range([0, 800]);
  const yScale = d3.scaleLinear().domain([0, 30]).range([400, 0]);

  const line = d3.line()
    .x(d => xScale(d.date))
    .y(d => yScale(d.value));

  svg.append("path")
    .datum(data)
    .attr("d", line)
    .attr("fill", "none")
    .attr("stroke", "blue");
</script>
```

4. Graphiques à bulles (Bubble Charts)

Les graphiques à bulles sont utilisés pour représenter des données multidimensionnelles où chaque bulle peut avoir des coordonnées X et Y, ainsi qu'une taille et une couleur qui représentent des données supplémentaires.

Propriétés des Graphiques à Bulles :

- `cx`, `cy` : Les coordonnées X et Y du centre de la bulle.
- `r` : Le rayon de la bulle, qui est souvent utilisé pour représenter une dimension supplémentaire (par exemple, la population d'une ville, la valeur d'une variable).
- `fill` : La couleur de la bulle, utilisée pour représenter une autre dimension des données (par exemple, la catégorie).

Exemple d'un Graphique à Bulles :

```
<svg width="600" height="600"></svg>
<script>
  const data = [
    {x: 100, y: 200, radius: 30, color: 'red'},
    {x: 200, y: 150, radius: 50, color: 'blue'},
    {x: 300, y: 300, radius: 20, color: 'green'},
    {x: 400, y: 100, radius: 40, color: 'yellow'}
  ];

  const svg = d3.select("svg");

  svg.selectAll("circle")
    .data(data)
    .enter()
    .append("circle")
    .attr("cx", d => d.x)
    .attr("cy", d => d.y)
    .attr("r", d => d.radius)
    .attr("fill", d => d.color);
</script>
```

---

5. Graphiques de Chord (Diagrammes de Cordes)

Les **diagrammes de cordes** sont utilisés pour représenter des relations entre plusieurs éléments, souvent sous forme de matrice. Ils sont utiles pour visualiser des flux, des connexions ou des interactions.

Propriétés des Diagrammes de Cordes :

- `d3.chord()` : Crée les liens entre les groupes.
- `d3.arc()` : Permet de dessiner les arcs pour les groupes du diagramme.
- `d3.ribbon()` : Permet de dessiner les cordes (liens entre les groupes).

Exemple d'un Diagramme de Cordes :

```
<svg width="600" height="600"></svg>
<script>
  const matrix = [
    [11975, 5871, 8916, 2860],
```

```
    [690, 5671, 4158, 3534],
    [7881, 3925, 7105, 1974],
    [6114, 3242, 2133, 7000]
  ];

  const colors = d3.scaleOrdinal(d3.schemeCategory10);

  const chord = d3.chord()
    .padAngle(0.05)
    .sortSubgroups(d3.descending);

  const arc = d3.arc()
    .innerRadius(200)
    .outerRadius(220);

  const ribbon = d3.ribbon()
    .radius(200);

  const svg = d3.select("svg")
    .append("g")
    .attr("transform", "translate(300,300)");

  const chords = chord(matrix);

  svg.selectAll(".group")
    .data(chords.groups)
    .enter()
    .append("path")
    .attr("d", arc)
    .style("fill", d => colors(d.index))
    .style("stroke", "#000");

  svg.selectAll(".ribbon")
    .data(chords)
    .enter()
    .append("path")
    .attr("d", ribbon)
    .style("fill", d => colors(d.target.index))
    .style("stroke", "#000");
</script>
```

---

6. Graphiques de Sankey (Diagrammes de Sankey)

Les diagrammes de **Sankey** sont utilisés pour représenter des flux (énergie, argent, etc.) entre différentes catégories. Ils permettent de visualiser l'intensité des flux à l'aide de largeurs proportionnelles aux valeurs.

Propriétés des Diagrammes de Sankey :

- `d3.sankey()` : Crée la mise en page du diagramme de Sankey. La largeur des flux est proportionnelle à la valeur des données.
- `nodeWidth()` : Définit la largeur des nœuds dans le diagramme.

- `nodePadding()` : Définit l'espacement entre les nœuds.

Exemple d'un Diagramme de Sankey :

```
<svg width="600" height="400"></svg>
<script>
  const data = {
    "nodes": [
      { "name": "A" },
      { "name": "B" },
      { "name": "C" }
    ],
    "links": [
      { "source": 0, "target": 1, "value": 10 },
      { "source": 1, "target": 2, "value": 5 },
      { "source": 0, "target": 2, "value": 7 }
    ]
  };

  const svg = d3.select("svg");
  const width = +svg.attr("width");
  const height = +svg.attr("height");

  const sankey = d3.sankey()
    .nodeWidth(20)
    .nodePadding(10)
    .size([width, height]);

  const graph = sankey(data);

  svg.append("g")
    .selectAll(".node")
    .data(graph.nodes)
    .enter()
    .append("rect")
    .attr("class", "node")
    .attr("x", d => d.x0)
    .attr("y", d => d.y0)
    .attr("width", d => d.x1 - d.x0)
    .attr("height", d => d.y1 - d.y0)
    .style("fill", "#aaa");

  svg.append("g")
    .selectAll(".link")
    .data(graph.links)
    .enter()
    .append("path")
    .attr("class", "link")
    .attr("d", d3.sankeyLinkHorizontal())
    .style("stroke", "#000")
    .style("stroke-width", d => Math.max(1, d.width));
</script>
```

---

7. Cartes Choroplèthes (Choropleth Maps)

Les **cartes choroplèthes** sont utilisées pour visualiser des données géographiques avec des couleurs représentant des valeurs associées à des zones géographiques (pays, régions, etc.).

Propriétés des Cartes Choroplèthes :

- `d3.geoPath()` : Permet de dessiner les formes géographiques sur une carte.
- `d3.geoMercator()` : Projette les coordonnées géographiques (latitude, longitude) en coordonnées pixel (x, y) pour les cartes.
- `colorScale()` : Une échelle utilisée pour colorier les différentes zones selon des valeurs numériques.

Exemple d'une Carte Choroplèthe :

```
<svg width="800" height="500"></svg>
<script>
  const width = 800;
  const height = 500;

  const projection = d3.geoMercator().scale(100).translate([width / 2, height /
1.5]);
  const path = d3.geoPath().projection(projection);

  d3.json("https://d3js.org/us-10m.v1.json").then(function(us) {
    const colorScale = d3.scaleQuantize().domain([0,
100]).range(d3.schemeBlues[9]);

    const svg = d3.select("svg");

    svg.selectAll("path")
      .data(topojson.feature(us, us.objects.states).features)
      .enter()
      .append("path")
      .attr("d", path)
      .style("fill", d => colorScale(d

.properties.value));
  });
</script>
```

8. Graphiques en Radar (Radar Charts)

Les graphiques en radar, également appelés graphiques en toile d'araignée, sont utiles pour afficher plusieurs variables quantitatives sur un axe circulaire. Ils sont souvent utilisés pour afficher des performances ou des comparaisons entre plusieurs entités sur diverses dimensions.

Propriétés des Graphiques en Radar :

- `d3.line()` : Utilisé pour connecter les points qui forment les axes du graphique.
- `scaleLinear()` : Permet de définir l'échelle de chaque dimension.
- `angle` : Utilisé pour positionner les axes autour du cercle.

Exemple d'un Graphique en Radar :

```
<svg width="600" height="600"></svg>
<script>
  const data = [
    { axis: "A", value: 10 },
    { axis: "B", value: 8 },
    { axis: "C", value: 7 },
    { axis: "D", value: 6 },
    { axis: "E", value: 5 },
  ];

  const width = 600, height = 600;
  const radius = Math.min(width, height) / 2;
  const angleSlice = Math.PI * 2 / data.length;

  const svg = d3.select("svg")
    .attr("width", width)
    .attr("height", height)
    .append("g")
    .attr("transform", `translate(${width / 2},${height / 2})`);

  const scale = d3.scaleLinear().domain([0, 10]).range([0, radius]);

  const radarLine = d3.lineRadial()
    .angle((d, i) => i * angleSlice)
    .radius(d => scale(d.value));

  svg.append("path")
    .datum(data)
    .attr("d", radarLine)
    .attr("fill", "rgba(0,0,0,0.2)")
    .attr("stroke", "#4D89F9");

  svg.selectAll(".axis")
    .data(data)
    .enter()
    .append("line")
    .attr("x1", 0)
    .attr("y1", 0)
    .attr("x2", (d, i) => scale(d.value) * Math.cos(i * angleSlice - Math.PI / 2))
    .attr("y2", (d, i) => scale(d.value) * Math.sin(i * angleSlice - Math.PI / 2))
    .attr("stroke", "gray");
</script>
```

---

9. Résolution des problèmes avec D3.js

Problèmes courants et solutions :

- **Problème 1 : Optimisation de la performance avec de grands ensembles de données**
  Utilisez la méthode `d3.forceSimulation()` pour les graphiques de réseaux et `d3.scaleBand()` pour les graphiques à barres. Assurez-vous que l'actualisation des éléments n'affecte pas la performance.

- **Problème 2 : Ajouter des animations fluides**
  Utilisez `transition()` pour ajouter des animations fluides lors de l'ajout, la suppression ou la mise à jour d'éléments SVG.

- **Problème 3 : Gérer les événements interactifs**
  Pour des interactions avancées, combinez `mouseover`, `mouseout`, `click` avec des **transitions** pour ajouter des effets visuels comme le changement de couleur ou le zoom.

Conclusion

Avec D3.js, vous pouvez non seulement créer des graphiques avancés mais également les enrichir d'interactions, d'animations et de visualisations dynamiques qui augmentent l'engagement et l'efficacité de vos utilisateurs. La clé réside dans une bonne compréhension des principes de base des graphiques D3, que vous pouvez ensuite étendre et adapter à vos besoins spécifiques.